双私钥结构

把整个 ATASSES 的内层加密系统抽象为**存在两把隐式的全局总私钥 S_1 和 S_2**，是一个极其深刻且直观的视角！

在这个视角下，两把总私钥分工明确：

- **第一总私钥 S_1（负责“拉格朗日插值通道”）**：由所有在线参与者 $i \in \mathcal{T}$ 的第一把内层私钥 $ek_{i,1}$ 经**拉格朗日加权**合成，即 $S_1 = \sum_{i \in \mathcal{T}} L_i^{(\mathcal{T})} \cdot ek_{i,1}$。它专门用来解开“乘了拉格朗日系数的解密份额 $\sum L_i^{(\mathcal{T})} b_i$。
- **第二总私钥 S_2（负责“直接累加通道”）**：由所有在线参与者 $i \in \mathcal{T}$ 的第二把内层私钥 $ek_{i,2}$ 经**等权重（系数为 1）直接相加**合成，即 $S_2 = \sum_{i \in \mathcal{T}} ek_{i,2}$。它专门用来解开“未乘拉格朗日系数的混淆噪声 $\sum n_i$。

而聚合器最终恢复出的那把总解密钥匙 dk，本质上就是这两把总私钥的**叠加态：$dk = S_1 + S_2$**！

下面，我们完全采用“双全局总私钥 (S_1, S_2) 协同演化”的全新方式，重新为你做一次严丝合缝的闭环推导。

基于“双总私钥 S_1, S_2”视角的完整闭环推导

符号约定：

外层业务 FHE 的全局总私钥记为 S_{outer}，每个参与者 P_i 持有的外层 Shamir 私钥份额为 s_i，满足 $\sum_{i \in \mathcal{T}} L_i^{(\mathcal{T})} s_i = S_{outer}$；外层待解密密文为 $CT = (c_0, c_1)$，外层部分解密份额为 $b_i = c_1 \cdot s_i \pmod{Q}$，满足 $\sum_{i \in \mathcal{T}} L_i^{(\mathcal{T})} b_i = c_1 \cdot S_{outer} = b$。内层加密模数为 Q'，缩放因子 $\Delta' = \lfloor Q'/Q \rfloor$。）

阶段 1：参与者本地初始化双通道子私钥与双通道密文（`PartyR1`）

为了在后续动态构造出 S_1 和 S_2，每个参与者 P_i 在本地建立两个平行的加密通道()：

1. **生成双通道本地子私钥**：
   - 通道 1（插值通道）子私钥：$ek_{i,1} \xleftarrow{\$} \chi'_{sk}$
   - 通道 2（累加通道）子私钥：$ek_{i,2} \xleftarrow{\$} \chi'_{sk}$
2. **分发子私钥的 Shamir 碎片**：P_i 构造两个 $T-1$ 次随机多项式 $f_{i,1}(X)$（常数项为 $ek_{i,1}$）和 $f_{i,2}(X)$（常数项为 $ek_{i,2}$），向每个参与者 P_j 点对点发送求值碎片()：

3. $\begin{cases} ekShare_{j,i,1} = f_{i,1}(\alpha_j) \\ ekShare_{j,i,2} = f_{i,2}(\alpha_j) \end{cases}$

4. **双通道平行加密（共用同一公共基底 $p'_1 u' = -a'u'$）：**
   - **通道 1 加密解密份额 b_i**（目标：后续让它乘 $L_i^{(\mathcal{T})}$）：
   - $CTb_i = \begin{cases} c'_{b,i,0} = (a' \cdot ek_{i,1}) \cdot u' + \Delta' \cdot b_i + e'_{i,1} \\ c'_1 = -a' \cdot u' \end{cases} \pmod{Q'}$
   - **通道 2 加密混淆噪声 n_i**（目标：后续让它直接相加）：
   - $CTn_i = \begin{cases} c'_{n,i,0} = (a' \cdot ek_{i,2}) \cdot u' + \Delta' \cdot n_i + e'_{i,2} \\ c'_1 = -a' \cdot u' \end{cases} \pmod{Q'}$
   - **上传**：P_i 将双通道密文 (CTb_i, CTn_i) 上传给服务器（聚合器）()。

阶段 2：服务器在密文域分别合成“通道 1 总密文”与“通道 2 总密文”

服务器收集到第一回合在线集合 $\mathcal{T}$（$|\mathcal{T}| \geq T$）发来的双通道密文，计算出拉格朗日系数 $\{L_i^{(\mathcal{T})}\}_{i\in\mathcal{T}}$ 并广播给所有参与者()。同时，服务器在本地将两个通道的密文分别聚合：

1. **合成通道 1 总密文 $CT_{total}^{(1)}$（执行拉格朗日加权）**()：服务器对所有 $c'_{b,i,0}$ 乘以 $L_i^{(\mathcal{T})}$ 后求和：

2. $$\begin{aligned} C_{total,0}^{(1)} &= \sum_{i\in\mathcal{T}} L_i^{(\mathcal{T})} \cdot c'_{b,i,0} \\ &= (a' \cdot u') \cdot \underbrace{\left(\sum_{i\in\mathcal{T}} L_i^{(\mathcal{T})} \cdot ek_{i,1}\right)}_{\text{隐式形成第一总私钥 } S_1} + \Delta' \cdot \underbrace{\left(\sum_{i\in\mathcal{T}} L_i^{(\mathcal{T})} \cdot b_i\right)}_{=b=c_1\cdot S_{outer}} + \underbrace{\sum_{i\in\mathcal{T}} L_i^{(\mathcal{T})} \cdot e'_{i,1}}_{E'_1} \\ &= (a' \cdot u') \cdot S_1 + \Delta' \cdot b + E'_1 \pmod{Q'} \end{aligned}$$

3. *物理意义*：此时通道 1 的总密文 $C_{total,0}^{(1)}$ 已经把碎片还原成了外层真实解密值 b，但它被**第一总私钥** $S_1 = \sum_{i\in\mathcal{T}} L_i^{(\mathcal{T})} ek_{i,1}$ 牢牢锁住了！

4. **合成通道 2 总密文 $CT_{total}^{(2)}$（执行等权直接相加）**：服务器对所有 $c'_{n,i,0}$ 直接求和（系数全为 1）：

5. $$\begin{aligned} C_{total,0}^{(2)} &= \sum_{i\in\mathcal{T}} c'_{n,i,0} \\ &= (a' \cdot u') \cdot \underbrace{\left(\sum_{i\in\mathcal{T}} ek_{i,2}\right)}_{\text{隐式形成第二总私钥 } S_2} + \Delta' \cdot \underbrace{\left(\sum_{i\in\mathcal{T}} n_i\right)}_{\text{温和总混淆噪声 } N_{sm}} + \underbrace{\sum_{i\in\mathcal{T}} e'_{i,2}}_{E'_2} \\ &= (a' \cdot u') \cdot S_2 + \Delta' \cdot N_{sm} + E'_2 \pmod{Q'} \end{aligned}$$

6. *物理意义*：此时通道 2 的总密文 $C_{total,0}^{(2)}$ 聚合了未被放大的温和总噪声 $N_{sm}=\sum_{i\in\mathcal{T}} n_i$，且它被**第二总私钥** $S_2=\sum_{i\in\mathcal{T}} ek_{i,2}$ 锁住了！
7. **为什么服务器不能分别解开 $C_{total,0}^{(1)}$ 和 $C_{total,0}^{(2)}$？** 如果参与者分别把 S_1 和 S_2 恢复给服务器，服务器就会单独解出精确的 b（泄露外层私钥）和单独的噪声 N_{sm}，协议直接崩溃()！ 因此，服务器必须将两个通道的总密文**同态相加（融合成一体）**：
8. $$\begin{aligned} C'_{all,0} &= C_{total,0}^{(1)} + C_{total,0}^{(2)} \\ &= (a'\cdot u')\cdot(\mathbf{S_1}+\mathbf{S_2}) + \Delta'\cdot(b+N_{sm}) + (E'_1+E'_2) \pmod{Q'} \end{aligned}$$
9. 配合公共第二项 $C'_{all,1}=-a'\cdot u'$，现在服务器只需要拿到**两把总私钥的和** $dk=S_1+S_2$（且绝不能单独知道 S_1 或 S_2），就能一次性解出绑定在一起的 $(b+N_{sm})$！

阶段 3：参与者本地构造 S_1 与 S_2 的份额并捆绑上传（`PartyR2`）

第二回合在线的任意参与者 P_j（属于集合 $\mathcal{T}_2$，$|\mathcal{T}_2|\geq T$）收到 $\{L_i^{(\mathcal{T})}\}_{i\in\mathcal{T}}$ 后，利用阶段 1 收到的碎片，在本地实际上是先算出了 S_1 的份额和 S_2 的份额：

1. **计算第一总私钥 S_1 的 Shamir 份额 $S_{1,j}$：**
2. $$S_{1,j}=\sum_{i\in\mathcal{T}} L_i^{(\mathcal{T})}\cdot ekShare_{i,j,1}=\sum_{i\in\mathcal{T}} L_i^{(\mathcal{T})} f_{i,1}(\alpha_j)$$
3. *(注：该式是合并多项式 $F_1(X)=\sum_{i\in\mathcal{T}} L_i^{(\mathcal{T})} f_{i,1}(X)$ 在 α_j 处的求值，其常数项 $F_1(0)=\sum_{i\in\mathcal{T}} L_i^{(\mathcal{T})} ek_{i,1}=S_1$。)*
4. **计算第二总私钥 S_2 的 Shamir 份额 $S_{2,j}$：**
5. $$S_{2,j}=\sum_{i\in\mathcal{T}} ekShare_{i,j,2}=\sum_{i\in\mathcal{T}} f_{i,2}(\alpha_j)$$
6. *(注：该式是合并多项式 $F_2(X)=\sum_{i\in\mathcal{T}} f_{i,2}(X)$ 在 α_j 处的求值，其常数项 $F_2(0)=\sum_{i\in\mathcal{T}} ek_{i,2}=S_2$。)*
7. **在本地将两把总私钥的份额“焊死”在一起：P_j 不单独发送 $S_{1,j}$ 或 $S_{2,j}$，而是将它们相加，生成联合密钥份额 $dkShare_j$ 并上传给服务器()：**
8. $$dkShare_j=S_{1,j}+S_{2,j}$$

阶段 4：服务器插值恢复联合总私钥 $dk=S_1+S_2$ 并完成解密（`AggregatorR2`）

1. **拉格朗日插值恢复 S_1+S_2：** 服务器收集到集合 $\mathcal{T}_2$ 发来的至少 T 个 $dkShare_j$，执行 Shamir 拉格朗日插值（系数记为 $L_j^{(\mathcal{T}_2)}$）：

2. $$
\begin{aligned}
dk &= \sum_{j\in\mathcal{T}_2} L_j^{(\mathcal{T}_2)} \cdot dkShare_j \\
&= \sum_{j\in\mathcal{T}_2} L_j^{(\mathcal{T}_2)} \cdot (S_{1,j} + S_{2,j}) \\
&= \underbrace{\sum_{j\in\mathcal{T}_2} L_j^{(\mathcal{T}_2)} S_{1,j}}_{=F_1(0)=S_1} + \underbrace{\sum_{j\in\mathcal{T}_2} L_j^{(\mathcal{T}_2)} S_{2,j}}_{=F_2(0)=S_2} \\
&= S_1 + S_2
\end{aligned}
$$

3. 服务器由此得到了联合总私钥 $dk = S_1 + S_2$，但由于 S_2 是随机生成的掩码密钥，服务器无法从 $S_1 + S_2$ 中剥离出单独的 S_1 或 S_2 ()。

4. **用 $dk = S_1 + S_2$ 解开融合总密文 CT_{all}**： 服务器执行内层线性解密()：

5. $$
\begin{aligned}
\mu' &= C'_{all,0} + C'_{all,1} \cdot dk \\
&= [(a' \cdot u') \cdot (S_1 + S_2) + \Delta' \cdot (b + N_{sm}) + (E'_1 + E'_2)] + (-a' \cdot u') \cdot (S_1 + S_2) \\
&= \Delta' \cdot (b + N_{sm}) + (E'_1 + E'_2) \pmod{Q'}
\end{aligned}
$$

6. **内层取整与外层最终解密**：
   - **内层取整（消去 $E'_1 + E'_2$）**：因为 $\|E'_1 + E'_2\| < \Delta'/2$，服务器除以 Δ' 并四舍五入，得到外层近似解密项()：
   - $$b' = \left\lfloor \frac{\mu'}{\Delta'} \right\rceil \pmod{Q} = b + N_{sm} = c_1 \cdot S_{outer} + \sum_{i\in\mathcal{T}} n_i \pmod{Q}$$
   - **外层合成与取整（消去 $e_{CT} + N_{sm}$）**：服务器将 b' 与外层密文首项 c_0 相加并除以 Δ 取整，由于外层总噪声 $\|e_{CT} + \sum_{i\in\mathcal{T}} n_i\| < \Delta/2$，最终无损恢复出业务明文 M ()：
   - $$M = \left\lfloor \frac{c_0 + b'}{\Delta} \right\rceil = \left\lfloor \frac{\Delta \cdot M + e_{CT} + \sum_{i\in\mathcal{T}} n_i}{\Delta} \right\rceil \pmod{P}$$

总结：为什么“双总私钥 (S_1, S_2)”的视角极具启发性？

用 S_1 和 S_2 来理解，整个方案的架构美感跃然纸上：

1. **双轨制并行处理**：系统实际上开辟了两条轨道——轨道 1（由 S_1 保护）专门负责跑拉格朗日插值 $\sum L_i^{(\mathcal{T})} b_i$ 以恢复秘密；轨道 2（由 S_2 保护）专门负责跑直接加法 $\sum n_i$ 以注入防泄漏噪声。

2. **密钥捆绑防止拆解**：如果让服务器分别拿到 S_1 和 S_2，服务器就能只解轨道 1 而不解轨道 2，从而绕过噪声保护。参与者在本地计算 $dkShare_j = S_{1,j} + S_{2,j}$ 的操作，本质上是在 Shamir 份额层面对两把总私钥实施了**不可逆的密码学焊接（Key Binding）**，逼迫服务器只能用 $S_1 + S_2$ 去解开已经融为一体的 $C_{total,0}^{(1)} + C_{total,0}^{(2)}$